

OBJECT ORIENTED PROGRAMMING USING C++



Study Material

Graphs:

- A Graph is a collection of vertices (nodes) and edges (connections) that link pairs of vertices.
- Types of Graphs:
 - Undirected Graph: Edges have no direction, meaning connections are bidirectional.
 - Directed Graph (Digraph): Each edge has a direction, pointing from one vertex to another.
- Graph Representation:
 - Adjacency Matrix
 - Adjacency List

Weighted Graphs:

- A Weighted Graph assigns a weight (or cost) to each edge.
- This weight can represent distance, time, or any cost factor associated with that connection.
- **Use Cases:**
 - Shortest Path Problems (e.g., Dijkstra's algorithm)
 - Network Flows
- **Example:**
 - Vertices representing cities and edges representing roads with distances as weights.

Directed Graphs (Digraphs):

- In a Directed Graph, each edge has a direction, indicated by an arrow. This implies that the connection flows only in one direction (from one vertex to another).
- **Use Cases:**
 - One-way roads between cities.
 - Task Scheduling in project management (dependencies between tasks).
- **Example:**
 - A directed edge from A to B represents a path from vertex A to vertex B, but not vice versa.

Graph Traversal:

- Graph Traversal:
- A process of visiting all the nodes (vertices) and edges in a graph.
- Two Primary Traversal Techniques:
 - Breadth First Search (BFS)
 - Depth First Search (DFS)

Breadth First Search (BFS):

- BFS is a traversal technique that explores the graph level by level. It starts from a source node and visits all its neighbors before moving to the next level of nodes.
- **Key Characteristics:**
 - Uses a queue to track nodes.
 - Traverses all nodes at the current level before moving to the next level.
- **Algorithm:**
 - Start from the source node.
 - Mark the node as visited and enqueue it.
 - Dequeue a node, visit its unvisited neighbors, and enqueue them.
 - Repeat until the queue is empty.

Breadth First Search (BFS) Example:

- **Steps in BFS:**
 - Start at A, visit B, C.
 - From B, visit D, E.
 - From C, visit F.
- **Example BFS Traversal (Undirected Graph):**
 - Consider the following graph:



Example:

```
void BFS(int start, int graph[][MAX], int n) {  
    int visited[MAX] = {0}, queue[MAX], front = 0, rear = 0;  
    printf("%d ", start);  
    visited[start] = 1;  
    queue[rear++] = start;  
  
    while (front < rear) {  
        int u = queue[front++];  
        for (int v = 0; v < n; v++) {  
            if (graph[u][v] && !visited[v]) {  
                printf("%d ", v);  
                visited[v] = 1;  
                queue[rear++] = v;  
            }  
        }  
    }  
}
```


Depth First Search (DFS):

- DFS is a traversal technique that explores a graph by going as deep as possible along each branch before backtracking.
- **Key Characteristics:**
 - Uses a stack or recursion to track nodes.
 - Visits nodes by diving into a branch, backtracks when it reaches a dead end, and moves to another branch.
- **Algorithm:**
 - Start from the source node.
 - Mark the node as visited and recursively explore its unvisited neighbors.
 - If a node has no unvisited neighbors, backtrack to the previous node and explore other branches.

Depth First Search (DFS) Example:

- Steps in DFS:
- Start at A, visit B, then D.
- Backtrack to B, visit E.
- Backtrack to A, visit C, then F.
- Example DFS Traversal (Undirected Graph):
- Consider the following graph:



Example:

```
void DFS(int v, int graph[][MAX], int visited[], int n) {  
    visited[v] = 1;  
    printf("%d ", v);  
  
    for (int u = 0; u < n; u++) {  
        if (graph[v][u] && !visited[u]) {  
            DFS(u, graph, visited, n);  
        }  
    }  
}
```

Comparison: BFS VS DFS:

- BFS (Breadth First Search):
 - Explores neighbors level by level.
 - Uses a queue (FIFO).
 - Suitable for shortest path algorithms.
- DFS (Depth First Search):
 - Explores as deep as possible in each branch before backtracking.
 - Uses a stack (LIFO) or recursion.
 - Suitable for exploring entire paths or connected components.